Anexo C —— Configuración del Entorno de Desarrollo Software.

C.1. Introducción y Requisitos.

El entorno de desarrollo software del NEORV32 requiere de un entorno de compilación cruzada capaz de traducir el código fuente en lenguaje C a instrucciones máquina de la arquitectura RISC-V (RV-32).

Para este proyecto se ha seleccionado el *toolchain* xPack GNU RISC-V Embedded GCC, en su versión precompilada con licencia MIT. La propia guía de usuario del procesador recomienda su descarga.

El proceso de instalación de la cadena de herramientas se ha llevado a cabo sobre el sistema operativo Ubuntu 24.04.02 LTS ejecutado en una máquina virtual, pero todos los pasos son válidos para cualquier sistema operativo basado en Linux.

C.2. Instalación del Toolchain RISC-V.

La distribución empleada en este proyecto es la xPack GNU RISC-V Embedded GCC (versión 14.2.0). Los pasos a seguir para su descarga e instalación son los siguientes:

1. Desde el terminal de Linux, creamos la carpeta donde se almacenará el paquete:

```
~$ mkdir -p ~/Descargas/riscv-toolchain
```

2. Se accede a la carpeta creada:

```
$ cd ~/Descargas/riscv-toolchain/
```

3. Se procede a la descarga ejecutando el comando:

```
~/Descargas/riscv-toolchain$ wget https://github.com/xpack-dev-tools/riscv-none-elf-gcc-xpack/releases/download/v14.2.0-1/xpack-riscv-none-elf-gcc-14.2.0-1-linux-x64.tar.gz
```

4. Se descomprime el paquete con el siguiente comando (puede tardar unos minutos):

```
~/Descargas/riscv-toolchain$ tar -xvf xpack-riscv-none-elf-gcc-14.2.0-1-linux-x64.tar.gz
```

5. El paquete descomprimido debe moverse a la dirección `/opt/riscv` (se crea automáticamente con el comando):

```
~/Descargas/riscv-toolchain$ sudo mv xpack-riscv-none-elf-gcc-14.2.0-1 /opt/riscv
```

6. Después, acceder a la dirección `/opt/riscv` y mostar el contenido:

```
~/Descargas/riscv-toolchain$ cd /opt/riscv

/opt/riscv$ ls
bin  distro-info  include  lib  lib64  libexec  README.md  riscv-none-elf  share
```

7. Para que el compilador pueda utilizar el *framework* del NEORV32, el sistema necesita poder acceder a él, para ello se ejecuta el comando:

```
/opt/riscv$ echo 'export PATH=$PATH:/opt/riscv/bin' >> ~/.bashrc
```

8. De este modo el sistema tendrá acceso siempre al *toolchain*. Para que surja efecto se tiene que recargar el contenido del archivo `.bashrc`:

```
/opt/riscv$ source ~/.bashrc
```

9. Para comprobar que la instalación ha ido correctamente se puede ejecutar el siguiente comando. La respuesta deseada se muestra en la Fig.1.

```
/opt/riscv$ riscv-none-elf-gcc -version
```

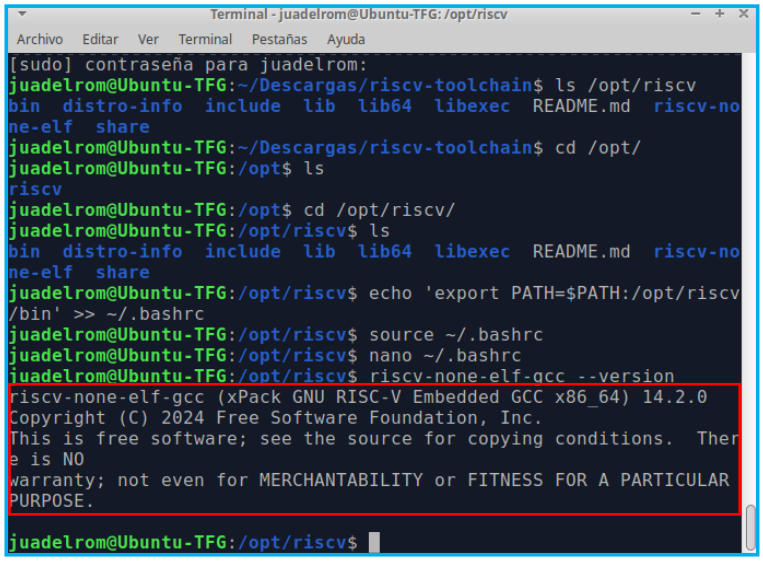

Fig.1. Salida del terminal con la versión instalada de GNU RISC-V Embedded GCC.

C.3. Descarga e Instalación del Framework del NEORV32.

En este apartado se abordarán los pasos necesarios para descargar e integrar el *framework* del procesador. Este posee todos los archivos necesarios para la implementación hardware y software, simulación y compilación de proyectos con el NEORV32.

Los pasos a seguir se detallan a continuación:

1. En primer lugar, si no se tiene instalada la herramienta Git para clonar repositorios:

```
~$ sudo apt update && sudo apt install git -y
```

2. Se clona el repositorio oficial mediante el comando:

```
~$ git clone https://github.com/stnolting/neorv32.git
```

3. Tras esto, se puede acceder a la raíz del proyecto:

```
~$ cd neorv32/
```

4. En el momento de clonar el repositorio, Git mostrará la versión más reciente del mismo. Dado que la versión empleada en este trabajo ha sido la v1.11.2, se deberá ejecutar:

```
~/neorv32$ git checkout v1.11.2
```

5. Para comprobar el funcionamiento del *framework* y la correcta integración del *toolchain* en el proyecto, se compilará el programa de usuario del ejemplo `demo_blink_led` alojado en la carpeta de ejemplos (`neorv32/sw/example`). Se accede a la dirección del ejemplo:

```
~/neorv32$ cd sw/example/demo_blink_led
```

6. Posteriormente, se ejecuta el comando make para compilar el archivo fuente (`main.c`) en un ejecutable binario (`neorv32_exe.bin`):

```
~/neorv32/sw/example/demo_blink_led$ make clean_all exe
```

Nota: si al ejecutar se produce un error del tipo:

```
make clean_all exe
No se ha encontrado la orden «make», pero se puede instalar con:
sudo apt install make        # version 4.3-4.1build1, or
```

```
sudo apt install make-guile  # version 4.3-4.1build1
```

O bien:

```
~/neorv32/sw/example/demo_blink_led$ make clean_all exe
Memory utilization:
   text       data     bss     dec     hexfilename
    836          0       0     836     344main.elf
Compiling image generator...
make: gcc: No existe el archivo o el directorio
make: *** [../../../sw/common/common.mk:168: ../../../sw/image_gen/image_gen] Error
127
```

Se necesita instalar la herramienta de automatización make, el compilador GCC de C nativo de Linux, o ambos. Para solucionar los dos problemas se puede ejecutar:

```
sudo apt update && sudo apt install build-essential -y
```

7. Si todo ha ido, bien debería haberse generado el archivo `neorv32_exe.bin` (Fig.2.).

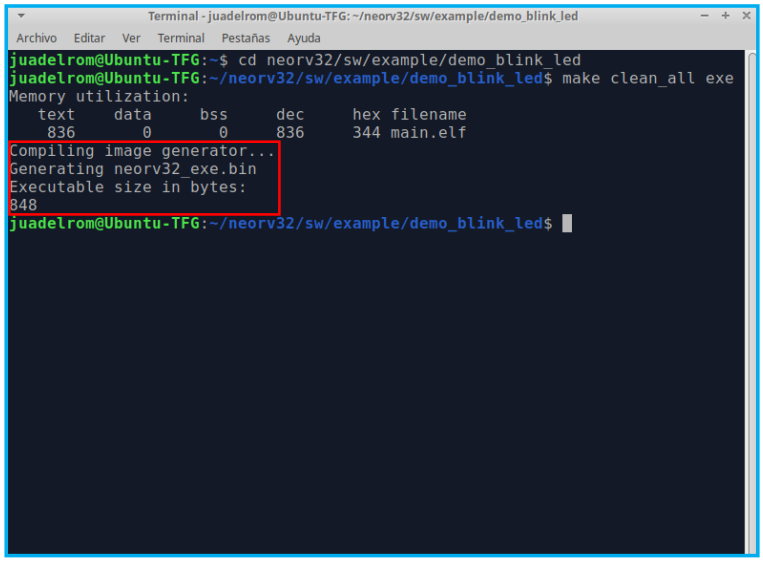

Fig.2. Salida del terminal con la generación exitosa del archivo binario.